

Course Outline: Measuring Frontend Performance: Core Web Vitals

Title: Measuring Frontend Performance: Core Web Vitals **Skill:** 35 — Debugging and optimizing the frontend **Learnpack:** + Midiendo el desempeño del frontend (Core Web Vitals) **File:** s35-w12d36-midiendo-el-desempeno-del-frontend-core-web-vitals **Prerequisite:** Docker Compose (Skill 34) **Target Audience:** Beginner developers who have built full-stack applications and want to measure and improve their frontend performance **Total Lessons:** 7 **Format:** Conceptual (0% hands-on)

Course Description

You've built full-stack applications — React frontends, FastAPI backends, containerized with Docker. But how fast are they? This course introduces Core Web Vitals, Google's three key metrics for measuring real-world frontend performance. You'll learn what FCP, LCP, and TBT measure, what causes poor scores, and how to fix them. By the end, you'll be able to run Lighthouse on any web app and interpret the results with confidence.

Course Philosophy

This course emphasizes:

- Measurement before optimization — "you can't improve what you don't measure"
 - Understanding root causes, not just memorizing fixes
 - Connecting metrics to real user experience, not abstract scores
-

Course Statistics Summary

Section	Lessons
00 - Welcome	1
01 - Core Web Vitals	4
02 - Assessment	1
03 - Outro	1
Total	7

00.0 Is Your Frontend Fast Enough?

Learning Objectives:

- Understand why frontend performance directly affects user experience and business outcomes
- Know the measurement-optimization mindset: "you can't improve what you don't measure"

- Preview the three Core Web Vitals (FCP, LCP, TBT) covered in this course
- Know the primary tool for measuring CWV: Lighthouse

Content Outline:

- The performance gap: identical-looking apps can have wildly different speeds
- Why it matters beyond aesthetics: bounce rates, SEO rankings, accessibility on slow connections
- The measurement-first mindset: performance optimization without measurement is guesswork
- What Lighthouse is and how to run it (Chrome DevTools → Lighthouse tab)
- Overview of the three metrics: FCP, LCP, TBT — each measures a different dimension of performance
- Course roadmap: one section, one deep-dive per metric

Transitions: This welcome lesson sets up the core question — "is my app fast?" — and introduces Lighthouse as the answer. Section 01 opens by explaining what Core Web Vitals are before zooming into each metric individually.

Key Principles:

- Performance is a feature, not an afterthought
- "You can't improve what you don't measure" — every optimization effort starts with a baseline score
- Core Web Vitals are grounded in real-world user experience data, not synthetic benchmarks

Examples:

- Amazon's research: every 100ms of latency costs approximately 1% in sales
 - Google's 3-tier scoring: Good / Needs Improvement / Poor — clear, actionable thresholds
 - A Lighthouse report showing three scores side by side for a real-world site
-

01.0 Core Web Vitals

Learning Objectives:

- Define Core Web Vitals and explain why Google introduced them
- Know the two dimensions CWV measures: loading (FCP, LCP) and interactivity (TBT)
- Know the three measurement tools: Lighthouse, PageSpeed Insights, Chrome DevTools Performance tab
- Understand the Good / Needs Improvement / Poor threshold system

Content Outline:

- What are Core Web Vitals?
 - A set of metrics defined by Google to measure real-world user experience on the web
 - Part of Google's Page Experience ranking signals — poor CWV can hurt SEO
 - Not invented arbitrarily: derived from large-scale data on what correlates with user satisfaction
- The two dimensions of frontend performance captured by CWV:
 - Loading: does the content appear quickly? (FCP, LCP)

- Interactivity: is the page responsive after content appears? (TBT)
- Measurement tools:
 - Lighthouse: built into Chrome DevTools, simulates a page load and reports all three metrics
 - PageSpeed Insights: Google's online tool — combines lab data (Lighthouse) with real-world field data (CrUX)
 - Chrome DevTools Performance tab: records and shows a flame chart of exactly what ran and when
- The scoring system:
 - Each metric has its own thresholds for Good, Needs Improvement, and Poor
 - Green = Good, Orange = Needs Improvement, Red = Poor
- Section preview:
 - 01.1 dives into FCP — the first signal that anything is loading
 - 01.2 covers LCP — the signal that the main content has arrived
 - 01.3 examines TBT — the signal that the page is actually usable

Transitions: The welcome lesson introduced the question "is my app fast?". This lesson answers "what does Google measure to find out?" — setting up the three deep-dives that follow. After this lesson, students understand the framework; the next three lessons zoom in on each metric.

Key Principles:

- CWV are Google's opinionated answer to "what actually affects users?" — three metrics, each measuring something the others miss
- Lighthouse is the starting point for any CWV optimization effort
- Lab metrics (Lighthouse) and field metrics (PageSpeed Insights CrUX data) are complementary — lab shows what's fixable, field shows what real users experience

Examples:

- Running Lighthouse on a popular e-commerce site and identifying which metrics are red
- A PageSpeed Insights report showing the difference between lab scores and real-world field data
- Two identical-looking pages: one scores 95 in Lighthouse, one scores 32 — the difference is entirely invisible to the eye

01.1 First Contentful Paint

Learning Objectives:

- Define FCP and describe precisely what it measures
- Know Google's FCP thresholds: Good $\leq 1.8s$, Needs Improvement $\leq 3s$, Poor $> 3s$
- Identify the most common causes of slow FCP
- Know the key strategies to improve FCP scores

Content Outline:

- What FCP measures: the time from page navigation start to the moment the browser renders the first piece of DOM content (text, image, SVG, non-white canvas element)

- FCP ≠ page fully loaded — it's just "something appeared"
- Before FCP: the user sees a blank white screen
- After FCP: the user knows loading is happening
- Why FCP matters: it's the first signal that the page is alive — it eliminates the "is this broken?" anxiety
- Thresholds:
 - Good: ≤ 1.8 seconds
 - Needs Improvement: 1.8s – 3s
 - Poor: > 3s
- Common causes of slow FCP:
 - Render-blocking CSS: the browser cannot paint until all CSS in `<head>` is fully parsed
 - Render-blocking JavaScript: synchronous scripts in `<head>` pause HTML parsing
 - Slow Time to First Byte (TTFB): if the server takes 2s to respond, FCP can't start for 2s
 - Unoptimized web fonts: if the font file hasn't loaded, text may not render (invisible text)
- How to improve FCP:
 - Inline critical CSS (the styles needed for above-the-fold content) — load the rest asynchronously
 - Add `defer` or `async` to JavaScript that doesn't need to run before render
 - Optimize server response time (caching, CDN, faster database queries)
 - Use `font-display: swap` so text renders immediately in a fallback font while the web font loads

Anti-pattern — Loading All CSS in One Large Bundle:

- Why it's tempting: simple to maintain, all styles in one file
- What goes wrong: a 300KB CSS bundle blocks all rendering until fully downloaded and parsed — on a slow connection, FCP is 4s+ even for a simple page
- Correct approach: identify the critical CSS needed for the initial viewport, inline it in `<head>`, and load the full stylesheet asynchronously after the page renders

Transitions: The previous lesson (01.0) introduced FCP as one of two loading metrics. This lesson defines it precisely and explains its causes. The next lesson (01.2) builds on the same question — "when does content appear?" — but focuses on the *most important* content rather than the first pixel.

Key Principles:

- FCP is the earliest user-visible feedback that a page is loading — every millisecond before FCP is a blank screen
- Render-blocking resources are the primary enemy of FCP
- FCP can be fast even if the main content hasn't loaded yet — that's exactly what LCP measures

Examples:

- A news site: FCP fires when the first headline or navigation item appears (even before the article content loads)
- Lighthouse "Opportunities" section: "Eliminate render-blocking resources" is the most common FCP recommendation
- Before/after: adding `defer` to a 150KB analytics script → FCP drops from 3.2s to 1.4s

01.2 Largest Contentful Paint

Learning Objectives:

- Define LCP and explain how it differs from FCP
- Know Google's LCP thresholds: Good $\leq 2.5s$, Needs Improvement $\leq 4s$, Poor $> 4s$
- Identify what elements the browser considers as LCP candidates
- Know the key strategies to improve LCP scores

Content Outline:

- What LCP measures: the time until the largest above-the-fold content element finishes rendering
 - LCP candidates: `<img>` elements, `<video>` poster images, elements with a CSS background image, and large block-level text nodes
 - The browser tracks all candidates and reports the largest one as the LCP element
 - LCP can change during page load (a larger element may render after a smaller one)
- LCP vs FCP: the critical distinction
 - FCP = something appeared (could be a loading spinner, a nav bar, a single pixel)
 - LCP = the main content appeared (the hero image, the article headline, the product photo)
 - FCP answers "did loading start?" — LCP answers "did the user get what they came for?"
- Thresholds:
 - Good: ≤ 2.5 seconds
 - Needs Improvement: $2.5s - 4s$
 - Poor: $> 4s$
- Common causes of slow LCP:
 - Unoptimized hero images: large file sizes (1MB+ JPEGs) on the most prominent element
 - Late image discovery: the LCP image is in CSS or loaded via JavaScript, so the browser finds it late
 - Client-side rendering: with a React SPA (no SSR), the HTML is nearly empty — the browser must download JS, execute it, and build the DOM before anything renders
 - Slow CDN or missing CDN: image served from a distant origin server adds hundreds of ms
- How to improve LCP:
 - Compress and resize hero images to the actual display size (not original resolution)
 - Use modern formats: WebP or AVIF provide 25–50% smaller files at equivalent quality
 - Add `<link rel="preload" as="image">` for the LCP image — tells the browser to fetch it immediately, before it discovers it in the HTML
 - Consider Server-Side Rendering (SSR) or static generation for content-heavy pages
 - Deliver images from a CDN close to the user

Anti-pattern — Using the Original High-Resolution Photo as a Hero Image:

- Why it's tempting: "higher resolution looks better, and storage is cheap"
- What goes wrong: a 3.5MB PNG hero image → LCP of 7s+ on a mobile connection, Poor score, Google SEO penalty
- Correct approach: export at the display dimensions (e.g., 1200×600px), compress to WebP, target under 200KB for a hero image

Transitions: 01.1 covered FCP (the first paint). This lesson covers LCP (the meaningful paint). Together, FCP and LCP answer the question "how quickly does the page look complete?". The next lesson (01.3) shifts focus entirely: the page may look complete, but is it actually usable?

Key Principles:

- LCP is Google's proxy for "perceived load complete" — the moment the user feels the page has arrived
- The LCP element is almost always the hero image or the primary headline — optimize these first
- Image optimization is consistently the highest-leverage LCP improvement

Examples:

- E-commerce product page: the LCP element is the main product image — if it's a 2MB JPEG, LCP will be Poor
 - Blog post: LCP is typically the featured image or the article title (whichever is larger)
 - Before/after: converting a 1.8MB JPEG hero to 140KB WebP + preload → LCP drops from 5.1s to 1.9s (Good)
-

01.3 Total Blocking Time

Learning Objectives:

- Define TBT and understand the concept of the main thread and long tasks
- Know Google's TBT thresholds: Good $\leq 200\text{ms}$, Needs Improvement $\leq 600\text{ms}$, Poor $> 600\text{ms}$
- Identify the most common causes of high TBT
- Know the key strategies to reduce TBT
- Understand the relationship between TBT and INP

Content Outline:

- Background: the browser's main thread
 - The browser uses a single main thread to parse HTML, run JavaScript, and paint pixels
 - While JavaScript is executing, the browser cannot respond to user input (clicks, taps, typing)
 - This is why JS-heavy pages can look fully loaded but feel unresponsive
- What TBT measures:
 - Any task that runs on the main thread for more than 50ms is a "long task"
 - For each long task: blocking time = task duration – 50ms (the 50ms the browser can "afford")
 - TBT = the sum of all blocking times for long tasks between FCP and TTI (Time to Interactive)
 - Example: a 200ms task contributes 150ms to TBT; a 300ms task contributes 250ms
- Why TBT matters: it captures the "page looks loaded but doesn't respond" experience that FCP and LCP completely miss
- Thresholds:
 - Good: $\leq 200\text{ms}$
 - Needs Improvement: 200ms – 600ms

- Poor: > 600ms
- TBT vs INP:
 - TBT is a lab metric (measured by Lighthouse in a simulated environment)
 - INP (Interaction to Next Paint) is the field equivalent — measures real-world interaction responsiveness
 - Both are about JavaScript blocking the main thread; TBT is easier to reproduce and debug
- Common causes of high TBT:
 - Large JavaScript bundles: downloading and parsing 500KB+ of JS creates long tasks
 - Unoptimized third-party scripts: analytics, ads, chat widgets, and A/B testing tools often run during page load
 - Synchronous operations: blocking loops or heavy computation on the main thread
 - Excessive DOM manipulation: creating thousands of DOM nodes at once
- How to reduce TBT:
 - Code splitting: split the JS bundle so each page only loads the code it needs
 - Lazy loading: defer loading of non-critical components until the user needs them
 - Defer third-party scripts: load analytics and chat widgets after the page is interactive
 - Move heavy computation to Web Workers (run off the main thread)
 - Audit and remove unused JavaScript with the Chrome DevTools Coverage tab

Anti-pattern — Importing an Entire Utility Library for One Function:

- Why it's tempting: `import _ from 'lodash'` is one line and gives access to everything
- What goes wrong: lodash adds 70KB+ to the bundle; during page load, the browser parses and executes this entire file, creating a long task and contributing hundreds of ms to TBT
- Correct approach: import only the specific function (`import debounce from 'lodash/debounce'`) or use a lighter built-in alternative

Transitions: 01.1 and 01.2 covered loading (FCP and LCP). This lesson covers the third dimension: interactivity. Together, the three metrics describe the full user experience. The next lesson tests your understanding of all three.

Key Principles:

- The main thread can only do one thing at a time — long JS tasks starve the user of interactivity
- TBT exposes performance problems that are invisible to the eye but felt by every user who tries to click something
- JavaScript bundle size is the biggest lever for TBT; code splitting is the most impactful fix

Examples:

- A React SPA with a 600KB unoptimized bundle: FCP is fast (HTML loads), LCP is fast (HTML has text), but TBT is 900ms — buttons don't respond for nearly a second after the page looks ready
- Chrome DevTools Performance tab: red "long task" markers over the main thread flame chart
- Before/after: implementing route-based code splitting in a React app → TBT drops from 850ms to 140ms (Good)

Learning Objectives:

- Apply understanding of FCP, LCP, and TBT to real-world performance scenarios
- Diagnose performance problems from described symptoms
- Select correct optimization strategies for specific issues
- Interpret metric thresholds and Lighthouse scores accurately

Question Format: Scenario-based (10 questions)

Questions:

1. A developer runs Lighthouse on their portfolio site and gets FCP: 4.2s. Which of the following is the most likely cause?
 - A) The hero image is 3MB
 - B) There is a 200KB synchronous CSS file in the `<head>` blocking render
 - C) The page has too many DOM nodes
 - D) The LCP element is a `<video>` poster
 - **Correct: B** — Render-blocking CSS in `<head>` is the primary cause of slow FCP. The hero image affects LCP, not FCP.
2. A page loads visually in 1.5 seconds (good FCP), but buttons don't respond to clicks for 3 seconds after appearing. Which metric is most likely Poor?
 - A) FCP
 - B) LCP
 - C) TBT
 - D) TTFB
 - **Correct: C** — TBT measures main thread blocking during loading. The page looks ready (good FCP) but JavaScript is still running (high TBT).
3. What is the key difference between FCP and LCP?
 - A) FCP measures images; LCP measures text
 - B) FCP fires when any DOM content renders; LCP fires when the largest above-the-fold element renders
 - C) FCP is a field metric; LCP is a lab metric
 - D) FCP is measured in frames per second; LCP is measured in seconds
 - **Correct: B** — FCP = first anything; LCP = largest meaningful content.
4. A developer's Lighthouse report shows LCP: 5.8s (Poor). The LCP element is a hero image. What should they try first?
 - A) Add `defer` to all JavaScript files
 - B) Inline the critical CSS
 - C) Compress the hero image and convert it to WebP; add `<link rel="preload">` for it
 - D) Move the hero image to a Web Worker
 - **Correct: C** — LCP for a hero image is almost always fixed by image optimization (compression, modern format, preload).

5. A long task on the main thread runs for 320ms. How much does it contribute to TBT?

- A) 320ms
- B) 270ms
- C) 50ms
- D) 0ms — only tasks over 500ms count
- **Correct: B** — Blocking time = task duration – 50ms = 320 – 50 = 270ms.

6. A team is using `import moment from 'moment'` to format a single date. Which Core Web Vital metric will this most likely harm, and why?

- A) FCP — because moment.js blocks CSS parsing
- B) LCP — because moment.js delays the hero image
- C) TBT — because the large moment.js bundle creates long tasks on the main thread during JS execution
- D) None — JavaScript imports do not affect Core Web Vitals
- **Correct: C** — Large JS bundles are the primary cause of high TBT.

7. Which of the following elements can be the LCP candidate? (Select the best answer)

- A) An inline SVG icon in the navigation
- B) A `<p>` tag with a single sentence
- C) A large `<img>` hero image above the fold
- D) A CSS border
- **Correct: C** — LCP candidates are large images, video posters, background images, and large text blocks. A large hero image is the most common LCP element.

8. What is Google's "Good" threshold for LCP?

- A) ≤ 1.8 seconds
- B) ≤ 2.5 seconds
- C) ≤ 4 seconds
- D) ≤ 200 milliseconds
- **Correct: B** — LCP Good threshold is ≤ 2.5 s. (FCP Good is ≤ 1.8 s; TBT Good is ≤ 200 ms.)

9. A single-page React application has no server-side rendering. The HTML file is nearly empty and the app is built entirely by JavaScript. Which metrics are most likely to suffer?

- A) FCP and TBT only
- B) LCP and TBT — the main content appears late (LCP) and the JS execution creates long tasks (TBT)
- C) Only FCP — because the HTML is empty
- D) None — Core Web Vitals only apply to static sites
- **Correct: B** — Without SSR, the browser must download, parse, and execute JS before any content renders (slow LCP) and the large JS execution creates long tasks (high TBT).

10. A developer wants to improve TBT. Their app loads 8 third-party scripts (analytics, A/B testing, chat widget) synchronously during page load. What is the most targeted fix?

- A) Compress all images on the page
- B) Add `font-display: swap` to all web fonts
- C) Defer loading of third-party scripts until after the page is interactive
- D) Enable HTTPS
- **Correct: C** — Third-party scripts running synchronously during load create long tasks. Deferring them removes those tasks from the critical loading window, directly reducing TBT.

Evaluation Criteria:

- Correct identification of which metric is affected by a given symptom
- Accurate knowledge of FCP, LCP, and TBT thresholds
- Ability to connect root causes (large images, JS bundles, render-blocking CSS) to the correct metric
- Ability to select the most targeted fix for a described problem

Score Interpretation:

- 9–10 correct: Excellent — you understand Core Web Vitals deeply and can diagnose real-world performance issues
- 7–8: Good — minor gaps remain; revisit the metric(s) where you hesitated
- 5–6: Needs review — revisit the metric definitions, thresholds, and causes before moving on
- Below 5: Revisit the full course; focus on understanding what each metric measures and what causes poor scores

03.0 Measure, Improve, Repeat

Content Outline:

- Course recap: what you've accomplished
 - You can now define FCP, LCP, and TBT — and explain what each one means for real users
 - You know the Good thresholds for each metric and the most common causes of poor scores
 - You have a concrete toolkit: Lighthouse, PageSpeed Insights, Chrome DevTools Performance tab
- The measurement-optimization cycle: run Lighthouse → identify the worst metric → apply the targeted fix → re-run → repeat
- How CWV connects to the rest of your stack: image CDNs, SSR decisions, JS bundle strategy, font loading — all of these affect your scores
- Next steps to explore:
 - Run Lighthouse on a project you've built — start with a baseline score
 - Explore PageSpeed Insights with a real public URL to see field data (CrUX)
 - Investigate INP (Interaction to Next Paint) — the field equivalent of TBT, now part of Google's Core Web Vitals set
 - Try the Chrome DevTools Coverage tab to find unused JavaScript in your bundles
- Resources for continued learning:
 - web.dev/vitals — Google's official CWV documentation

- PageSpeed Insights — pagespeed.web.dev
- Chrome DevTools Performance documentation
- web.dev/articles/inp — understanding INP
- Encouragement: measurement is the beginning of all optimization. Every great-performing website started with someone running Lighthouse, seeing a red score, and deciding to fix it. You now have the vocabulary to do the same.

Note: This is conclusion only — no theory, exercises, or assessment.